

O código implementa o problema clássico da mochila 0/1 (knapsack), cujo objetivo é maximizar o valor total de itens sem ultrapassar a capacidade da mochila. Ele usa programação dinâmica com uma tabela bidimensional $dp[i][w]$, onde cada posição guarda o melhor valor possível usando os primeiros i itens e capacidade w . A transição compara não incluir o item atual ($dp[i-1][w]$) com incluí-lo (somando seu valor e reduzindo a capacidade). O algoritmo percorre iterativamente todos os itens e capacidades possíveis. O método principal testa a função com dois conjuntos de capacidade diferentes. Os principais conceitos Java usados são arrays, loops aninhados e programação dinâmica tabular. A complexidade de tempo é $O(n * \text{capacidade})$ e a de espaço também $O(n * \text{capacidade})$. Como limitação, o consumo de memória pode ser alto para capacidades grandes, podendo ser otimizado para $O(\text{capacidade})$ usando otimização de estado.